

CODING SKILL TREE

Dispensa progressiva di algoritmi e complessità

Dalle implementazioni non conformi agli invarianti efficienti

Manuale cumulativo per il ripasso offline dei pattern algoritmici emersi nelle quest settimanali e nelle relative review tecniche.

Giuseppe Sinatra

Edizione progressiva - aggiornata al 7 agosto 2026

Nota metodologica

Questa dispensa non è una raccolta generale di algoritmi. Ogni capitolo nasce da un'evidenza osservabile: un pattern nuovo affrontato in assessment, una soluzione non corretta oppure un'implementazione corretta sugli esempi ma non conforme ai vincoli di scala.

L'analisi segue una struttura costante: modello del problema, approccio iniziale, diagnosi, invariante dell'algoritmo efficiente, argomento di correttezza, complessità ed elementi utili al riconoscimento del pattern. I riquadri rossi conservano il valore diagnostico dell'implementazione iniziale; i riquadri verdi presentano la versione di riferimento validata in review.

Regola di lettura

Il codice rosso non è necessariamente errato negli output. Può essere semanticamente corretto e, al tempo stesso, inaccettabile rispetto ai limiti computazionali del problema. La distinzione tra correttezza funzionale ed efficienza asintotica è parte integrante della valutazione algoritmica.

Indice

Nota metodologica	ii
1 Predicati esatti e sliding window	1
1.1 Predicati esatti nell'aggregazione di record	1
1.1.1 Modello, ipotesi e notazione	1
1.1.2 Approccio iniziale ed errore logico	1
1.1.3 Algoritmo corretto e invariante	2
1.1.4 Correttezza, complessità ed edge case	2
1.2 Sliding window: modello del problema	3
1.3 Sliding window: approccio iniziale	4
1.3.1 Diagnosi tecnica	4
1.4 Sliding window: algoritmo efficiente	5
1.5 Sliding window: argomento di correttezza	5
1.6 Sliding window: analisi di complessità	6
1.7 Sliding window: edge case rilevanti	6
1.8 Sliding window: come riconoscere il pattern	6
1.9 Checklist di ripasso	7
Indice dei pattern	8

Predicati esatti e sliding window

Origine: Week 001, esercizi 2 e 4

Contesto: aggregazione di record e picco di richieste temporali

Esito della review: un errore logico e un vincolo di scala non rispettato

Pattern: predicati esatti; sliding window a due indici

1.1 Predicati esatti nell'aggregazione di record

1.1.1 Modello, ipotesi e notazione

Sia $R = (r_0, \dots, r_{n-1})$ una sequenza di record. Ogni record può contenere i campi `model`, `tokens` e `success`. Per ogni nome di modello valido occorre calcolare numero di run, numero di successi e somma dei token. La condizione sui successi è intenzionalmente stretta:

$$\text{successo}(r_i) \iff r_i[\text{success}] \text{ is True.}$$

Il contratto non afferma che il campo contenga esclusivamente booleani; pertanto non è lecito sostituire questa condizione con la truthiness.

1.1.2 Approccio iniziale ed errore logico

La submission costruisce correttamente un dizionario indicizzato per modello, gestisce token mancanti o nulli e non modifica gli input. La riga che calcola `successful_run`, tuttavia, accetta ogni oggetto truthy. Ad esempio, 1 e "yes" aumentano il contatore pur non essendo l'oggetto booleano `True`. Si tratta di un errore di correttezza, non di una semplice inefficienza.

Il codice seguente conserva integralmente la submission; sono omessi soltanto gli spazi finali invisibili, privi di effetto sull'esecuzione, per esigenze di presentazione tipografica.

Submission originale - errore logico sul predicato

```
1 def summarize_model_runs(runs):
2     """Aggrega run, successi e token per modello."""
3     lookup = {}
4     for record_or in runs:
5         record=record_or.copy()
6         model=record.get('model')
7         tokens= 0 if record.get('tokens') is None else record.get('tokens')
8         successful_run=1 if record.get("success") else 0
9         if model is None:
```

```

10         continue
11     if model not in lookup:
12         lookup[model]={"total_runs": 1, "successful_runs": successful_run, "
total_tokens": tokens}
13     else:
14         lookup[model]["total_runs"]+=1
15         lookup[model]["successful_runs"]+=successful_run
16         lookup[model]["total_tokens"]+=tokens
17
18     return lookup

```

1.1.3 Algoritmo corretto e invariante

La struttura di aggregazione resta appropriata. È sufficiente inizializzare le statistiche del modello, aggiornare esattamente una volta il numero di run e tradurre letteralmente il predicato con `is True`.

Invariante dell'aggregazione

Dopo avere elaborato i primi i record, per ogni modello m presente nel dizionario le tre statistiche sono esattamente quelle dei record validi tra r_0 e r_{i-1} con modello m ; in particolare, `successful_runs` conta soltanto i record il cui valore di `success` è l'oggetto `True`.

Implementazione di riferimento - predicato conforme

```

1 def summarize_model_runs(runs):
2     """Aggrega run, successi e token per modello."""
3     summary = {}
4     for record in runs:
5         model = record.get("model")
6         if model is None:
7             continue
8
9         if model not in summary:
10            summary[model] = {
11                "total_runs": 0,
12                "successful_runs": 0,
13                "total_tokens": 0,
14            }
15
16            stats = summary[model]
17            stats["total_runs"] += 1
18            stats["successful_runs"] += int(record.get("success") is True)
19            tokens = record.get("tokens")
20            stats["total_tokens"] += 0 if tokens is None else tokens
21
22    return summary

```

1.1.4 Correttezza, complessità ed edge case

L'invariante vale inizialmente perché il dizionario è vuoto. Ogni record senza modello valido viene ignorato come richiesto. Per un record valido, l'algoritmo aggiorna una sola voce: incrementa il

numero di run, aggiunge un successo se e solo se il predicato esatto è vero e somma zero oppure il numero di token. L'invariante è quindi preservato; dopo n iterazioni descrive l'intero input e implica la correttezza del risultato.

Il tempo è $O(n)$ in media, assumendo accessi hash $O(1)$, e lo spazio ausiliario è $O(k)$ per k modelli distinti. I casi da verificare includono lista vuota, modello mancante o `None`, nome di modello vuoto, token mancanti, nulli o uguali a zero, e valori `success` quali `False`, `None`, `0`, `1` e stringhe non vuote.

Segnale di riconoscimento

Espressioni come “soltanto quando”, “esattamente”, “è `True`” o “è `None`” richiedono di distinguere identità, uguaglianza e truthiness. Prima di codificare, trascrivere il predicato della specifica in una condizione booleana senza allargarne il dominio.

1.2 Sliding window: modello del problema

Sia

$$T = (t_0, t_1, \dots, t_{n-1})$$

una sequenza di timestamp ordinata in modo non decrescente e sia $w > 0$ la durata della finestra. Si vuole determinare il massimo numero di eventi contenuti in un intervallo semiaperto di ampiezza w :

$$M = \max_s |\{t_i \in T \mid s \leq t_i < s + w\}|.$$

La scelta dell'intervallo semiaperto è sostanziale: un evento posto esattamente in $s + w$ non appartiene alla finestra che inizia in s .

Precondizioni operative

La soluzione lineare presuppone timestamp già ordinati. Se l'ordinamento non è garantito dal contratto, occorre prima ordinare la sequenza, portando il costo complessivo a $O(n \log n)$. La durata deve essere strettamente positiva; in caso contrario la funzione solleva `ValueError`.

1.3 Sliding window: approccio iniziale

L'implementazione presentata durante la quest seleziona ogni timestamp come possibile inizio e, per ciascuno, visita nuovamente l'intera sequenza. Questa strategia è concettualmente semplice e restituisce gli output attesi, ma esegue fino a n^2 verifiche di appartenenza.

Implementazione iniziale - non conforme al vincolo di scala

```
1 def max_requests_in_window(timestamps, window_seconds):
2     """Restituisce il picco di richieste in una finestra temporale."""
3     count_max=0
4     if window_seconds <= 0:
5         raise ValueError("La finestra dev'essere un valore positivo")
6     for start in timestamps:
7         count=0
8         for input in timestamps:
9             if start <= input < start+window_seconds:
10                 count+=1
11             count_max=max(count, count_max)
12
13     return count_max
```

1.3.1 Diagnosi tecnica

Il problema non risiede nella condizione di appartenenza, che rappresenta correttamente l'intervallo semiaperto. L'inefficienza nasce dalla perdita di informazione tra due finestre consecutive: quando l'estremo destro avanza, gli eventi ancora validi vengono contati nuovamente da zero.

Inoltre, il nome locale `input` oscura la funzione built-in omonima di Python. Non altera il risultato in questo caso, ma riduce la chiarezza del codice e può generare errori in estensioni successive.

Con un limite di $n = 200\,000$, una crescita quadratica è incompatibile con un coding assessment ordinario:

$$n^2 = 40\,000\,000\,000$$

controlli potenziali. Il superamento dei test visibili certifica soltanto alcuni risultati, non il rispetto dei vincoli computazionali.

1.4 Sliding window: algoritmo efficiente

Poiché i timestamp sono ordinati, gli eventi validi per un dato estremo destro formano sempre un segmento contiguo della sequenza. È quindi sufficiente mantenere due indici:

- **right** introduce il nuovo evento corrente;
- **left** avanza soltanto quando l'evento più vecchio non rientra più nella finestra.

Invariante della finestra

Dopo il ciclo **while**, tutti e soli gli eventi con indice compreso tra **left** e **right** appartengono alla finestra semiaperta che termina sul timestamp corrente. In particolare,

$$t_{\text{right}} - t_{\text{left}} < w.$$

L'indice **left** è il più piccolo indice che rende vera questa disuguaglianza.

Implementazione di riferimento - complessità lineare

```
1 def max_requests_in_window(timestamps, window_seconds):
2     """Restituisce il picco di richieste in una finestra temporale."""
3     if window_seconds <= 0:
4         raise ValueError("La finestra dev'essere un valore positivo")
5
6     left = 0
7     best = 0
8
9     for right, current_time in enumerate(timestamps):
10         while current_time >= timestamps[left] + window_seconds:
11             left += 1
12
13         current_count = right - left + 1
14         best = max(best, current_count)
15
16     return best
```

1.5 Sliding window: argomento di correttezza

Si consideri un indice destro fissato. Prima dell'esecuzione del **while**, la finestra può contenere timestamp troppo vecchi. Ogni iterazione incrementa **left** finché il timestamp corrente non dista meno di w dal timestamp più vecchio conservato. Per ordinamento, tutti gli elementi successivi a **left** soddisfano allora la stessa condizione.

Alla terminazione del ciclo, **left** è minimo: se esistesse un indice valido più piccolo, il **while** si sarebbe fermato prima. Di conseguenza, $\text{right} - \text{left} + 1$ è il massimo numero di eventi in una finestra valida il cui evento più recente è t_{right} . Considerando in successione ogni possibile estremo destro e conservando il massimo, l'algoritmo restituisce M .

1.6 Sliding window: analisi di complessità

Strategia	Tempo	Spazio ausiliario
Scansione annidata	$O(n^2)$	$O(1)$
Sliding window	$O(n)$	$O(1)$
Ordinamento + sliding window	$O(n \log n)$	dipende dall'ordinamento

Il ciclo `while` annidato non rende quadratica la seconda soluzione. L'indice `left` non arretra mai: nel corso dell'intera esecuzione può avanzare al massimo n volte. Anche `right` percorre la sequenza una sola volta; il lavoro totale è quindi lineare.

1.7 Sliding window: edge case rilevanti

- **Sequenza vuota:** il ciclo non viene eseguito e il risultato è zero.
- **Timestamp duplicati:** restano simultaneamente nella finestra e vengono conteggiati come eventi distinti.
- **Evento sul bordo destro:** se la distanza è esattamente w , l'evento più vecchio viene escluso, coerentemente con $[s, s + w)$.
- **Durata non positiva:** il problema non definisce una finestra valida; la funzione segnala esplicitamente l'errore.
- **Input non ordinato:** l'invariante non è garantito; occorre ordinare oppure rifiutare l'input secondo il contratto.

1.8 Sliding window: come riconoscere il pattern

La sliding window è una candidata naturale quando il problema presenta contemporaneamente:

- una sequenza ordinata o ordinabile;
- un intervallo contiguo definito da una soglia;
- una statistica da massimizzare o minimizzare su tutti gli intervalli;
- finestre consecutive che condividono gran parte degli elementi.

Regola operativa per l'assessment

Quando due finestre consecutive differiscono per pochi elementi, chiedersi quale stato possa essere aggiornato incrementalmente. Se ogni elemento entra ed esce dalla struttura al massimo una volta, è spesso possibile ottenere una soluzione $O(n)$.

1.9 Checklist di ripasso

Prima di considerare concluso un problema analogo, verificare:

1. se l'ordinamento è garantito oppure deve essere costruito;
2. se gli estremi dell'intervallo sono inclusivi o esclusivi;
3. quale invariante deve valere dopo l'avanzamento di `left`;
4. quante volte ciascun indice può avanzare complessivamente;
5. se la complessità ottenuta è compatibile con la dimensione massima dichiarata.

Stato di apprendimento

Il pattern è stato introdotto e analizzato, ma non è ancora masterizzato. La mastery richiederà implementazioni lineari autonome, corrette e completate nel timebox su più problemi con formulazioni differenti.

Indice dei pattern

Pattern	Prima introduzione	Stato al capitolo
Predicati esatti	Week 001	Introdotta, da consolidare
Sliding window a due indici	Week 001	Introdotta, da consolidare